

Projet 2 MS14 : Algorithme de Delaunay et application à la compression d'image

Racky BA

April 19, 2026

Dans cette partie on s'intéresse à la compression d'une image en niveaux de gris en utilisant les propriétés d'interpolation et d'unicité de la triangulation de Delaunay en 2D. En effet, on part d'une image représentée dans un maillage structuré, dont chaque sommet a un niveau de gris (figure 1), que l'on souhaite compresser ou décompresser en représentant l'image sur un maillage de Delaunay. L'image initiale est représentée par une fonction u de $[1, m] \times [1, n]$, m, n étant les dimensions de l'image, et l'image compressée est représentée par une liste de pixels significatif (liste de sommets qu'on a choisis de garder), et leur niveau de gris. Cette représentation est suffisante puisque le maillage de Delaunay étant unique, on peut reconstruire la triangulation avec cette liste de points. On pourra à la fin estimer la qualité de la compression par le PSNR (peak signal to noise ratio). Pour établir toutes les étapes de la compression d'image, on va d'abord implémenter un noyau de Delaunay par la méthode de Bowyer-Watson et le tester, pour ensuite l'utiliser à proprement dit dans l'étape de compression d'image.

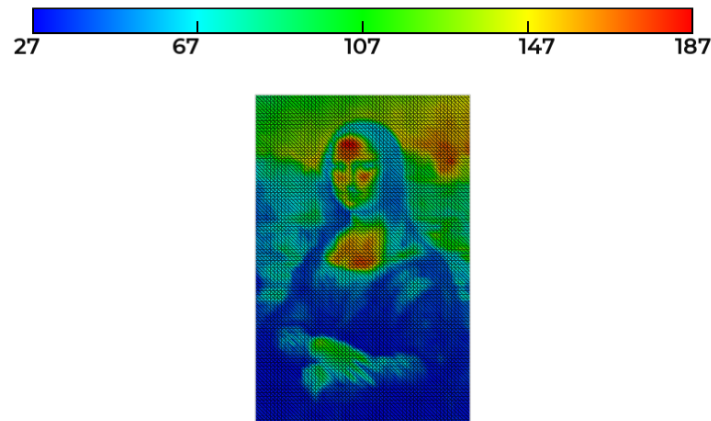


Figure 1: Image de la Joconde représentée par un maillage structuré, dont chaque sommet comporte un niveau de gris visualisé sur vizir4.

1 Noyau de Delaunay

Cette partie concerne l'écriture du noyau de Delaunay par la méthode de Bowyer-Watson qui peut se décomposer en 3 étapes :

- La localisation du point P à insérer.
- la création de la cavité C_P contenant les triangles dont le cercle circonscrit inclut P .
- , la retriangulation en étoilant la cavité avec le point P , et la mise à jour du tableau des voisins.

1.1 Localisation du point P

Pour localiser le point, P, on a utilisé la méthode de parcours local d'un triangle à son voisin pour de proche en proche arriver au triangle contenant P.

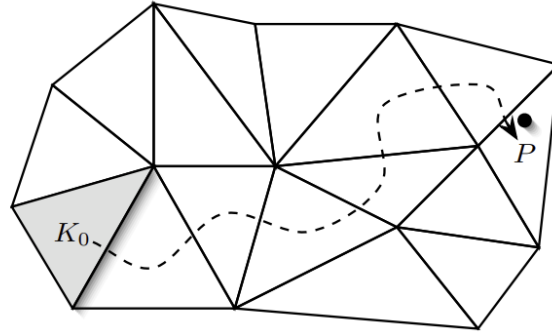


Figure 2: Algorithme de localisation par les voisins

Pour savoir où le point P se situe par rapport à un triangle donné, on calcule les coordonnées barycentriques du point P par rapport au triangle. Sur l'exemple de la figure 3, le triangle $K = P_0P_1P_2$ on a

$$P(x, y) = \sum_{i=0}^2 \beta_i(x, y) P_i$$

avec

$$\beta_0(x, y) = \frac{|PP_1P_2|}{|K|}, \quad \beta_1(x, y) = \frac{|P_0PP_2|}{|K|}, \quad \beta_2(x, y) = \frac{|P_0P_1P|}{|K|}$$

Ainsi on a plusieurs cas :

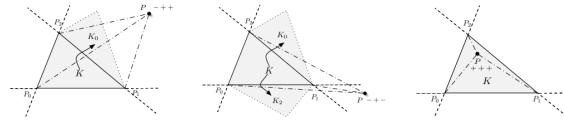


Figure 3: Exemple de configuration du point P par rapport au triangle K.

- Si $\beta_0, \beta_1, \text{ et } \beta_2$ sont tous entre 0 et 1 alors P est à l'intérieur du triangle.
- Si $\beta_i < 0$ alors P est du côté du voisin du triangle K par l'arête opposée à P_i . On a ainsi une direction pour se rapprocher du point P.
- Si on a deux coefficients $\beta < 0$ Il faut choisir un des deux triangles voisins pour poursuivre le parcours.

On a ainsi procédé de la manière suivante pour éviter de tomber sur un bord ou sur une boucle infinie de voisins.

1. Initialisation : On commence par le premier triangle sur la liste, et on calcule ses coefficients
2. On initialise une liste *triVisite* contenant tous les triangles explorés avec ce premier triangle.
3. Si les coefficients sont positifs, c'est gagné
4. Sinon, tant qu'au moins un coefficient est nul, on choisit un triangle voisin dont le coefficient est nul, en vérifiant que ce voisins existe.
5. Si le triangle n'est pas déjà visités, on mets à jours ses coefficients et on le marque visité. La boucle continue tant que tous les coefficients ne sont pas positifs ou tous les triangles non encore visités.

6. Si il est déjà visité c'est qu'on a fait une boucle et on risque de ne jamais s'arrêter. On va donc choisir un triangle non encore visité et continuer le processus.

Cet algorithme permet de sortir de cycle limite, mais pourrait ne pas trouver P sur un maillage dont l'enveloppe n'est pas convexe.

1.2 Création de la cavité C_p

Pour créer la cavité, on va itérativement rajouter des triangles dans la cavité en l'initialisant par le triangle contenant le Point P. Les éléments de la cavité sont représenté par une pile *PilElt* à deux curseurs, un *ptrNxt* pour empiler, donc rajouter un triangle à la pile, et un pointeur *ptrCur* pour dépiler, donc retirer un triangle. On a également la pile des arêtes du bord de la cavité *PilEdg*

1. Localisation de P et initialisation de la cavité

- 1a Localiser le point $P = iVer \rightarrow base[P] = iTri, \quad ptrCur = ptrNxt = 0$
- 1b Initialiser la cavité (i.e., la pile d'éléments) *PilElt* avec *iTri*, $ptrNxt \leftarrow ptrNxt + 1$
- 1c Initialiser le bord de la cavité (i.e., la pile d'arêtes) *PilEdg* avec e_0, e_1, e_2
- 2 Remplissage de la cavité Tant que la pile est non vide $ptrCur < ptrNxt$
- 2a Dépiler l'élément suivant $iTri = PilElt[ptrCur], ptrCur ++$
- 2b Boucle sur les 3 voisins $jTri = TriVoi[iTri][j]$
 - Vérifier si *jTri* est déjà dans la pile d'éléments
 - Si $jTri \in C_P$: (on a comparé le rayon du cercle circonscrit à jTri à la distance entre le centre de ce cercle et P)
 - Empiler *jTri* dans la pile d'éléments, $ptrNxt \leftarrow ptrNxt + 1$
 - Supprimer l'arête commune à *iTri* et *jTri* de la pile d'arêtes (on a remis en haut de la pile l'élément à supprimer et on le dépile)
 - Ajouter les deux autres arêtes de *jTri* à la pile d'arêtes si elles ne sont pas déjà présentes.

On a rajouté la liste *deleted* qui permet de marquer les triangles qu'on va supprimés, donc les triangles de la cavité. On obtient :

- la liste des éléments de la cavité
- la liste des arêtes formant le bord de la cavité

1.3 Retriangulation

3. Insertion du point et création des nouveaux éléments

- 3a Ajouter *P* à la liste des sommets, (réallocation si besoin de Crd)
- 3b Boucle sur les arêtes du bord de la cavité $e = P_{e1}P_{e2}$
 - Création du nouveau triangle $PP_{e1}P_{e2}$ (doit être positif si convention)
 - S'il existe encore un élément supprimé de la cavité $\implies$ remplacer
 - Sinon ajouter un nouveau triangle dans la liste des triangles
 - Mise à jour des voisins*

Mise à jour des voisins Pour la mise à jour des voisins on a crée la liste LstTri des voisins des anciens triangles de la cavité, et des nouveaux triangles de la cavité, et on essaie de créer une table de hashage seulement avec les éléments de cette liste comme dans *msh_neighbors*.

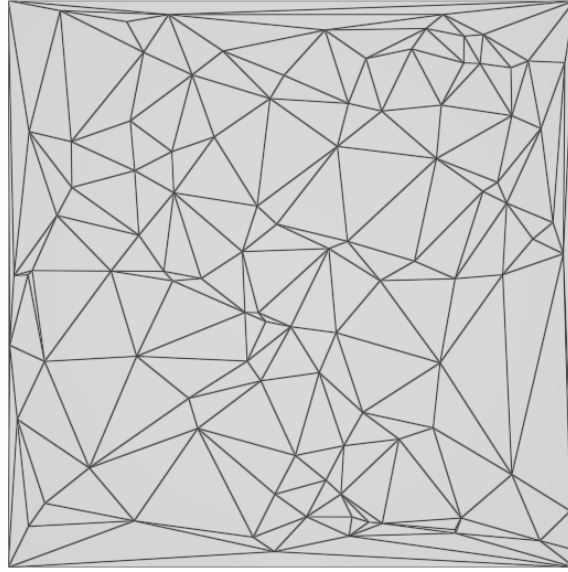


Figure 4: Insertion de 100 point de manière aléatoire avec le noyau de Delaunay sur un maillage de taille $[0, 1] \times [0, 1]$ avec 2 triangles

1.4 Test du noyau de Delaunay

On teste le noyau de Delaunay sur un domaine 2D qui sera le maillage vide comme le carré unité $[0, 1] \times [0, 1]$ défini par ses 4 sommets et 2 triangles. Ensuite à l'aide de la fonction `C rand()` on génère les coordonnées aléatoires des points qu'on va aller insérer, puis on trace la courbe de complexité donnant le temps CPU en fonction du nombre de points insérés (figure 5). On constate que cette courbe d'insertion se comporte en $n \log(n)$ donc qui est bien non quadratique car on ne reconstruit les voisins seulement des triangles affectés. On constate sur la figure 4, que le critère de Delaunay est

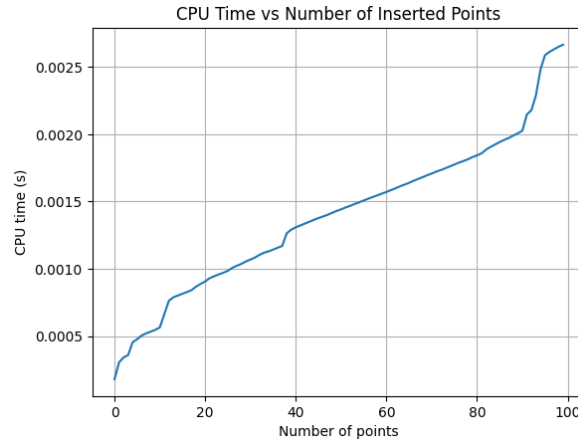


Figure 5: Temps cpu de l'insertion de 100 points aléatoires avec le noyau de Delaunay

respecté, à l'intérieur du maillage. Les triangles sont étirés aux bords car on est contraint par la taille de l'arête du bord, et on insère seulement à l'intérieur.

Ce test ne confirme que la bonne insertion des éléments internes. Si on veut insérer des éléments sur le bord, on a dû rajouter un test à l'insertion, vérifiant si le triangle qu'on va insérer n'est pas plat. À part cet aspect, l'insertion se fait avec la même méthode. Cependant, je n'ai pas réussi à faire fonctionner la mise à jour des voisins localement avec l'insertion sur des points du bord. Et j'ai donc remis tous les voisins du maillage globalement pour la suite par manque de temps.

2 Application à la compression d'image

Pour appliquer l'algorithme du noyau de delaunay à la compression d'image, on va d'abords créer un boîte englobante de la taille du maillage à compresser. Pour cela, on a utilisé le même carré que pour le test précédent, récupéré les coordonnées des coins du maillage à compresser et adapter les coordonnées des points du coin. Ensuite, l'algorithme de compression d'image se présente comme suit :

- Créer la bounding box
 - Initialiser le tableau sol et mettre les pixels des 4 coins.
 - (méthode naive) rajouter dans le tableau sol un pixel sur 5 avec le noyau de delaunay.
- Calcul du PSNR**
- Boucler sur les points du maillage à compresser
 - Les localiser dans le maillage compressé
 - Interpoler la valeur du pixel en ce point à partir des sommet du triangle. on obtient ainsi tous les pixels compressés $u_c(i, j)$ et on peut calculer le PSNR.

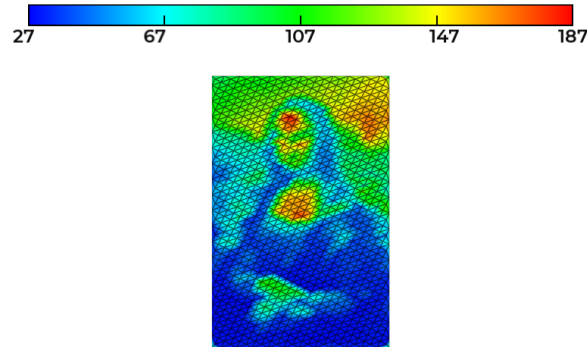


Figure 6: Compression naive de l'image de la joconde lowres ave $PSNR = 30,926$ et un facteur de compression $\alpha = \frac{1024}{2 \cdot 5104}$

On peut cependant faire mieux que cette méthode naive, par exemple, on peut choisir les pixels à conserver si l'erreur d'interpolation $u_c(i, j) - u(i, j)$ est plus petite qu'un certain seuil. Pour un seuil égal à 10, on obtient le résultat sur la figure 7 et on voit qu'on a amélioré le PSNR.

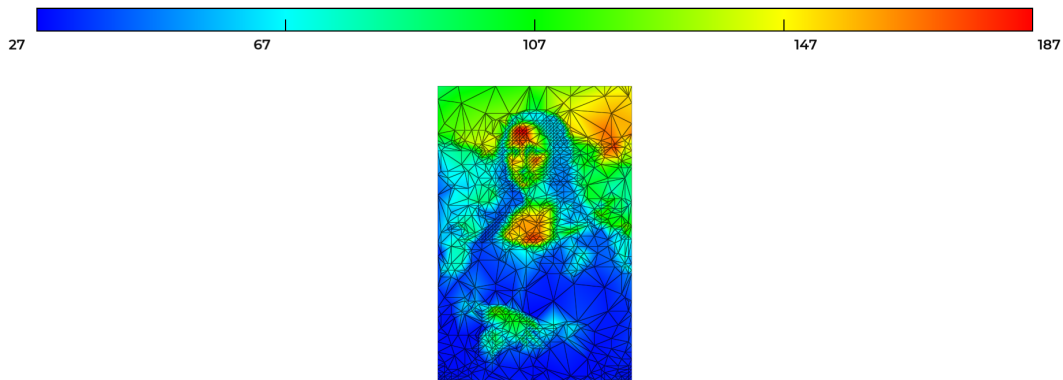


Figure 7: Compression de la joconde lowres en seuillant l'erreur d'interpolation avec $PSNR=32,709$ aet 1200 pixels

Remarque En essayant de compresser l'image haute résolution Joconde.mesh mon visual studio crash donc j'ai testé sur la version basse résolution uniquement.